

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



**BÁO CÁO
CƠ SỞ ĐO LƯỜNG VÀ ĐIỀU KHIỂN SỐ**

**BÀI TẬP 7
LẬP TRÌNH GIAO TIẾP I2C VÀ SPI GIỮA HAI VI ĐIỀU KHIỂN**

Nhóm 14: Lê Minh Quyền - 23020866

Vũ Hải Phong - 23020856

Phan Đình Phương Nam – 23020848

Tạ Đức Mạnh – 23020840

Giảng viên phụ trách - Phạm Duy Hưng

Hà Nội, năm 2026

I. TỔNG QUAN LÝ THUYẾT VỀ GIAO THỨC I2C VÀ SPI

1.1. Chuẩn giao tiếp I2C (Inter-Integrated Circuit)

1.1.1. Khái niệm và Cấu trúc tín hiệu phần cứng

Giao thức I2C do Philips Semiconductors phát triển là chuẩn giao tiếp đồng bộ (Synchronous) và bán song công (Half-duplex) hoạt động theo mô hình Master - Slave. Bus I2C chỉ sử dụng duy nhất 2 dây tín hiệu:

- **SDA (Serial Data):** Tuyến truyền dữ liệu hai chiều giữa Master và Slave.
- **SCL (Serial Clock):** Tuyến truyền xung nhịp đồng bộ, luôn do Master điều khiển.

Do cấu trúc các chân SDA và SCL của vi điều khiển là dạng cực máng hở (Open-Drain), chân chip chỉ có thể kéo bus xuống mức thấp (0V) chứ không thể tự đẩy lên mức cao. Do đó, bắt buộc phải kết nối hai điện trở kéo lên (Pull-up Resistor) từ SDA và SCL lên nguồn V_{CC} (thường chọn từ $2.2k\Omega$ đến $4.7k\Omega$) để duy trì trạng thái mặc định của bus là mức cao khi rảnh. Nếu chọn điện trở quá lớn, điện dung ký sinh đường dây sẽ làm méo dạng xung, gây lỗi dữ liệu ở tần số cao.

1.1.2. Cơ chế địa chỉ hóa và Khung truyền dữ liệu

I2C quản lý các Slave bằng địa chỉ phần mềm (thường là 7-bit). Khi Master khởi tạo phiên truyền, nó gửi địa chỉ lên bus; chỉ Slave có địa chỉ trùng khớp mới phản hồi. Một khung truyền I2C tiêu chuẩn diễn ra như sau:

1. **Điều kiện START:** Master kéo đường SDA từ cao xuống thấp khi SCL đang ở mức cao để báo động toàn bus.
2. **Truyền Địa chỉ & bit R/W:** Master gửi 7 bit địa chỉ của Slave mục tiêu, đi kèm bit thứ 8 là bit R/W (Read/Write) để xác định chiều truyền (0: Master ghi; 1: Master đọc).
3. **Bit ACK/NACK:** Tại chu kỳ xung SCL thứ 9, thiết bị nhận phải kéo SDA xuống thấp để phát tín hiệu ACK (xác nhận nhận tốt). Nếu SDA ở mức cao, đó là tín hiệu NACK (lỗi hoặc bận).
4. **Truyền dữ liệu:** Dữ liệu truyền theo từng byte (8-bit), bit MSB truyền trước, mỗi byte theo sau bởi 1 bit ACK/NACK.
5. **Điều kiện STOP:** Master chuyển đường SDA từ thấp lên cao khi SCL đang ở mức cao để kết thúc phiên truyền.

1.2. Chuẩn giao tiếp SPI (Serial Peripheral Interface)

1.2.1. Khái niệm và Cấu trúc tín hiệu phần cứng

Giao thức SPI do Motorola phát triển là chuẩn giao tiếp đồng bộ (Synchronous) và toàn song công (Full-duplex) tốc độ cao. SPI dựa trên kiến trúc thanh ghi dịch (Shift Register) liên kết khép kín, cho phép tại một chu kỳ xung nhịp, Master và Slave vừa truyền vừa nhận dữ liệu đồng thời. SPI sử dụng 4 dây tín hiệu:

- **MOSI (Master Out Slave In):** Đường truyền dữ liệu từ Master đến Slave.
- **MISO (Master In Slave Out):** Đường truyền dữ liệu từ Slave về Master.
- **SCK (Serial Clock):** Đường xung nhịp đồng bộ do Master tạo ra để điều khiển tốc độ dịch bit.

- **SS / CS (Slave Select):** Đường chọn Slave vật lý, mặc định tích cực mức thấp (0V).

SPI không dùng địa chỉ phần mềm mà quản lý bằng cơ chế chọn thiết bị phần cứng. Khi muốn giao tiếp với Slave nào, Master kéo đường SS của Slave đó xuống mức thấp. Các Slave không được chọn (SS ở mức cao) sẽ ngắt chân MISO của mình để tránh xung đột bus.

1.2.2. Các chế độ hoạt động (SPI Modes)

Sự đồng bộ trong SPI được định nghĩa qua hai tham số cấu hình: CPOL (Cực tính xung nhịp) và CPHA (Pha của xung nhịp), tạo nên 4 chế độ hoạt động tiêu chuẩn:

- **CPOL = 0 / 1:** Quy định đường SCK ở mức thấp / mức cao khi bus rảnh.
- **CPHA = 0 / 1:** Dữ liệu được lấy mẫu (Sample) tại cạnh đầu tiên / cạnh thứ hai của chu kỳ xung SCK.

Chế độ (Mode)	CPOL	CPHA	Trạng thái rảnh của SCK	Cạnh lấy mẫu dữ liệu
Mode 0	0	0	Mức thấp (0V)	Sườn lên (Cạnh đầu tiên)
Mode 1	0	1	Mức thấp (0V)	Sườn xuống (Cạnh thứ hai)
Mode 2	1	0	Mức cao (V_{CC})	Sườn xuống (Cạnh thứ hai)
Mode 3	1	1	Mức cao (V_{CC})	Sườn lên (Cạnh thứ hai)

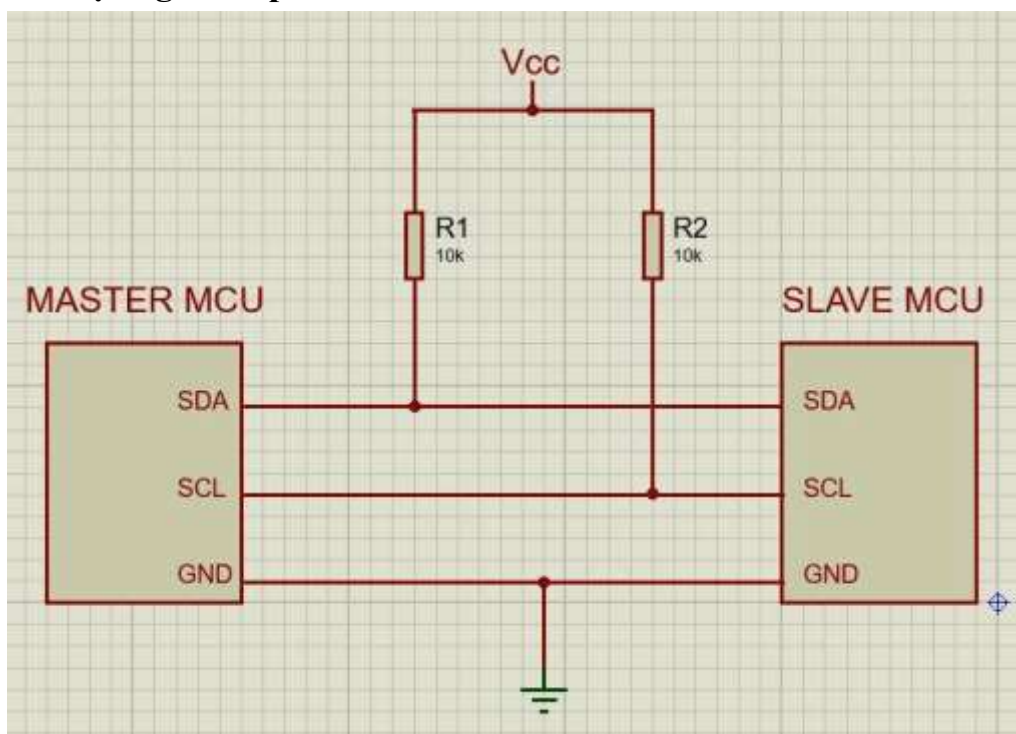
1.2.3. Phương thức truyền nhận dữ liệu

SPI truyền dữ liệu liên tục và **không có bit phản hồi phần cứng (No ACK)**. Tiến trình thực hiện:

1. Master kéo đường SS của Slave cần gọi xuống mức thấp (0V).
2. Master phát xung nhịp liên tục trên đường SCK.
3. Tại mỗi chu kỳ xung nhịp, Master dịch 1 bit ra đường MOSI, đồng thời Slave dịch 1 bit ra đường MISO để trao đổi dữ liệu cho đến khi đủ số byte cấu hình (gói 8-bit hoặc 16-bit).
4. Kết thúc gói tin, Master dừng phát xung SCK và kéo đường SS lên mức cao để giải phóng Slave.

II. THIẾT KẾ VÀ KẾT NỐI PHẦN CỨNG

2.1. Thiết kế mạch giao tiếp I2C



Mạch giao tiếp I2C

2.2. Thiết kế mạch giao tiếp SPI

Board MASTER (F411RE)	Chiều tín hiệu	Board SLAVE (F411RE)
PB5	→	PB12
PB13 (SCK)	→	PB13 (SCK)
PB15 (MOSI)	→	PB15 (MOSI)
PB14 (MISO)	←	PB14 (MISO)
GND	↔	GND

Mạch giao tiếp SPI

III. XÂY DỰNG CHƯƠNG TRÌNH LẬP TRÌNH

3.1. Kịch bản giao tiếp và Lưu đồ thuật toán

Để kiểm thử, nhóm xây dựng kịch bản: Vi điều khiển Master đọc một biến đếm liên tục thay đổi (giả lập dữ liệu cảm biến đo lường), sau đó đóng gói và truyền sang Slave. Slave nhận dữ liệu, kiểm tra lỗi và hiển thị trực quan.

3.1.1. Lưu đồ thuật toán phía Master

- **Bước 1:** Khởi tạo cấu hình hệ thống (Clock, GPIO, I2C, SPI).
- **Bước 2:** Cập nhật dữ liệu cần truyền (Biến đếm tăng tiến).
- **Bước 3 (Chế độ I2C):** Phát tín hiệu START → Gửi địa chỉ Slave + bit Write → Chờ ACK → Gửi dữ liệu → Phát tín hiệu STOP.
- **Bước 4 (Chế độ SPI):** Kéo chân SS xuống 0V → Phát xung SCK để dịch dữ liệu qua đường MOSI → Kéo chân SS lên 3.3V.
- **Bước 5:** Delay chu kỳ và quay lại Bước 2.

3.1.2. Lưu đồ thuật toán phía Slave

- **Bước 1:** Khởi tạo cấu hình ngoại vi ở chế độ Slave (Thiết lập địa chỉ đối với I2C).
- **Bước 2 (Chế độ I2C):** Lắng nghe bus → Nhận đúng địa chỉ thì trả mã ACK → Nhận dữ liệu lưu vào bộ đệm.
- **Bước 3 (Chế độ SPI):** Quét chân SS, nếu $SS = 0$ → Nhận xung SCK từ Master và dịch dữ liệu từ đường MOSI vào thanh ghi.
- **Bước 4:** Xử lý dữ liệu received, xuất kết quả ra LED/LCD và lặp lại.

IV. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1. Minh chứng kết quả hoạt động

Nhóm đã tiến hành nạp code vào hai vi điều khiển, kết nối phần cứng theo đúng sơ đồ thiết kế.

- **Kết quả kịch bản I2C và SPI:** Khi bấm nút thay đổi giá trị trên Master, dữ liệu dạng số được truyền đi lập tức. Màn hình hiển thị (hoặc trạng thái LED) phía Slave cập nhật giá trị chính xác theo thời gian thực, không xảy ra hiện tượng trễ hay mất gói tin.

4.2. Đánh giá sai số và Nhiễu đường truyền

- **Tính toàn vẹn dữ liệu:** Ở khoảng cách dây dẫn ngắn (< 20 cm), tỷ lệ lỗi bit (Bit Error Rate - BER) đo được trên cả hai chuẩn là 0%. Dữ liệu nhận hoàn toàn trùng khớp dữ liệu truyền.
- **Hiện tượng nhiễu thực tế:**
 - **Với I2C:** Khi thử nghiệm tăng tần số lên Fast Mode (400 kHz) với điện trở kéo lên lớn (10 k Ω), biên độ xung vuông bị suy giảm, sườn lên bị bo tròn do thời gian nạp xả của điện dung ký sinh trên dây dẫn không đủ nhanh. Lỗi này được khắc phục triệt để khi thay bằng điện trở 2.2 k Ω .

- **Với SPI:** Khi hoạt động ở tần số cao (vài MHz), hệ thống nhạy cảm với nhiễu xuyên âm (crosstalk) nếu các dây tín hiệu MOSI, MISO, SCK để quá sát nhau mà không có dây bọc chống nhiễu.

V. THẢO LUẬN VÀ SO SÁNH HAI CHUẨN GIAO TIẾP

Dựa trên quá trình thiết kế phần cứng và thực nghiệm lập trình, nhóm tổng hợp bảng so sánh định lượng và định tính giữa hai giao thức như sau:

Đặc điểm so sánh	Chuẩn giao tiếp I2C	Chuẩn giao tiếp SPI
Số dây tín hiệu	Ít (chỉ 2 dây SDA, SCL) → Tiết kiệm chân MCU.	Nhiều (Ít nhất 4 dây MOSI, MISO, SCK, SS).
Tốc độ truyền	Tốc độ trung bình (100 kbps - 400 kbps).	Tốc độ rất cao (Có thể đạt tới hàng chục Mbps).
Chế độ truyền	Bán song công (Half-Duplex).	Toàn song công (Full-Duplex).
Cơ chế quản lý	Dùng địa chỉ phần mềm → Thêm thiết bị không cần thêm chân dây quét.	Dùng chân chọn chip SS vật lý → Thêm 1 Slave phải tốn thêm 1 chân GPIO của Master.
Phản hồi phần cứng	Có bit ACK/NACK kiểm tra từng byte.	Không có cơ chế phản hồi (No ACK).
Phạm vi áp dụng	Thích hợp cho các linh kiện tốc độ thấp, cần tiết kiệm chân (Cảm biến, RTC, EEPROM).	Thích hợp cho các thiết bị cần truyền nhận khối dữ liệu lớn, tốc độ cao (Màn hình TFT, thẻ nhớ SD).

Link Code và video thực nghiệm:

https://github.com/Phong05-vn/csdl_I2C-SPI.git